INFORME CASO 3
Concurrencia y Sincronización de Procesos

ACLARACIÓN: Este informe es una versión resumida de lo que se pedía en el caso 3, no obstante , en la entrega realizada en github, se encuentra un análisis más a fondo de todo lo que se habla en este documento (se encuentra como ***informe_caso3.md*** en GitHub)

Integrantes
* Alejandro Cruz Acevedo – 201912149
* Nicolas Castano Calderon – 202420324

Tabla de Contenido

1. Introducción
2. Objetivo del proyecto
3. División del trabajo
4. Arquitectura general del sistema
5. Descripción de clases implementadas
6. Funcionamiento general del programa
7. Sincronización entre objetos
8. Validación y pruebas realizadas
9. Resultados obtenidos
10. Conclusiones

1. **Introducción**

En la actualidad, los sistemas distribuidos y los entornos basados en Internet de las Cosas (IoT) requieren procesar grandes volúmenes de información provenientes de múltiples dispositivos al mismo tiempo. Para resolver este tipo de situaciones, es necesario utilizar mecanismos de concurrencia que permitan la ejecución simultánea de tareas, garantizando al mismo tiempo consistencia, orden y coordinación entre los diferentes procesos.

El presente proyecto corresponde al desarrollo del Caso 3 de la asignatura Tecnologías e Infraestructura de Cómputo. En este caso se implementó una simulación de un sistema IoT dentro de un campus universitario, donde múltiples sensores generan eventos que deben pasar por diferentes etapas de análisis, filtrado, clasificación, consolidación y despliegue.

El principal propósito del trabajo fue aplicar conceptos de programación concurrente en Java mediante el uso de threads, monitores, sincronización con synchronized, espera pasiva con wait() y notifyAll(), espera semi-activa con Thread.yield(), y mecanismos de coordinación como CyclicBarrier y join().

2. **Objetivo del proyecto**

Diseñar e implementar una aplicación concurrente en Java capaz de simular el procesamiento completo de eventos generados por sensores IoT, garantizando:

* Ejecución paralela de múltiples actores del sistema.
* Sincronización correcta entre procesos que comparten recursos.
* Control del flujo de eventos mediante buzones acotados.
* Terminación ordenada de todos los hilos de ejecución.
* Validación del comportamiento mediante pruebas funcionales.

3. **División del trabajo**

El proyecto fue desarrollado en parejas y se dividió en dos bloques principales:

Bloque 1: Entrada y filtrado inicial

Encargado de recibir eventos y decidir su destino inicial.

Clases asociadas:

* Sensor
* BuzonEntrada
* Broker
* BuzonAlertas
* Administrador

Bloque 2: Clasificación y consolidación

Encargado de enrutar los eventos hacia los servidores finales y procesarlos.

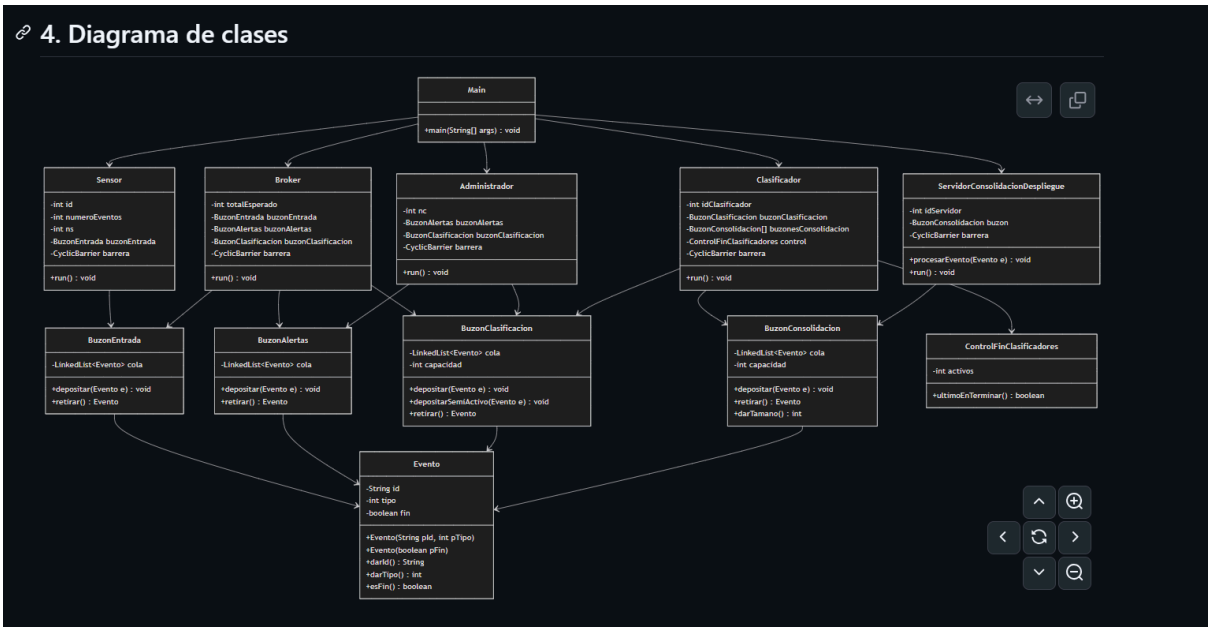
Clases asociadas:
* BuzonClasificacion
* Clasificador
* BuzonConsolidacion
* ServidorConsolidacionDespliegue
* ControlFinClasificadores

La parte desarrollada personalmente se concentró en el segundo bloque. El punto de integración entre ambas mitades del sistema es el `BuzonClasificacion`, donde llegan los eventos aprobados por el `Broker` y el `Administrador`, y desde donde son tomados por los clasificadores.

Se decidió que fuera de esta manera dado que BuzonClasificación era la conexión de estos grandes bloques, por lo que fue coherente tomar partes del trabajo de esta manera.
No obstante, se realizaron varias revisiones en cada bloque del compañero, haciendo casi juntos todo el código del caso.

4. **Arquitectura general del sistema**

En github se presenta de una manera rigurosa la arquitectura general del sistema:



Interpretación de la arquitectura

1. Los sensores generan eventos.
2. El bróker analiza cada evento.
3. Si el evento es normal, pasa a clasificación.
4. Si representa una alerta, pasa al administrador.
5. El administrador decide si lo descarta o lo reenvía.
6. Los clasificadores asignan cada evento al servidor correcto.
7. Los servidores procesan los eventos finales.

5. **Descripción de clases implementadas**

5.1 Clase Evento:
Representa la unidad básica de información del sistema.

Cada objeto contiene:

* Identificador del evento.
* Tipo del evento.
* Indicador de fin de ejecución.

Se implementaron dos clases lógicas de eventos:

Evento normal:
Contiene información real que debe ser procesada.

Evento de fin:
Se utiliza para indicar la terminación ordenada de clasificadores y servidores.

5.2 Clase BuzonEntrada:
Es una cola compartida donde los sensores depositan los eventos recién generados. El `Broker` consume desde este buzón.

5.3 Clase BuzonAlertas:
Almacena los eventos marcados por el bróker como posibles alertas. El administrador revisa estos eventos posteriormente.

5.4 Clase BuzonClasificacion:
Es un buzón acotado con capacidad `tam1`. Su función es recibir eventos listos para ser clasificados y distribuirlos a los threads clasificadores.

5.5 Clase BuzonConsolidacion:
Es un buzón acotado con capacidad `tam2`, asociado individualmente a cada servidor final.
Su función es desacoplar la velocidad de clasificación de la velocidad de procesamiento del servidor correspondiente.

5.6 Clase Sensor:
Cada sensor es un thread productor. Genera una cantidad determinada de eventos y los deposita en el buzón de entrada.

5.7 Clase Broker:
Consume eventos desde el buzón de entrada y decide si cada evento es normal o una alerta.

5.8 Clase Administrador:
Revisa alertas y decide si deben descartarse o reenviarse al buzón de clasificación.

5.9 Clase Clasificador:
Cada clasificador consume eventos desde el buzón de clasificación, identifica el tipo del evento y lo envía al buzón del servidor correspondiente.

5.10 Clase ServidorConsolidacionDespliegue:
Procesa eventos finales provenientes de su buzón de consolidación.

5.11 Clase ControlFinClasificadores:
Coordina qué clasificador fue el último en finalizar para que únicamente uno de ellos envíe los eventos de fin a los servidores.

5.12 Clase Main:
Es el punto de entrada del sistema. Lee la configuración, crea objetos, inicia los hilos y espera su terminación.

6. **Funcionamiento general del programa**

Paso 1. Lectura de configuración
Lo primero que realiza nuestro programa es que la clase principal lee el archivo *config.txt*, donde se definen parámetros como:

* número de sensores (*ni*)
* base de eventos (*base*)
* número de clasificadores (*nc*)
* número de servidores (*ns*)
* capacidad del buzón de clasificación (*tam1*)
* capacidad de los buzones de consolidación (*tam2*)

Paso 2. Creación de hilos y objetos compartidos
Se crean todos los buzones, sensores, bróker, administrador, clasificadores y servidores.

Paso 3. Inicio sincronizado
Mediante una CyclicBarrier, todos los hilos inician simultáneamente.

Paso 4. Producción de eventos
Cada sensor genera eventos y los deposita en BuzonEntrada.

Paso 5. Análisis inicial
El Broker consume eventos y decide:

* Evento normal → BuzonClasificacion
* Alerta → BuzonAlertas

Paso 6. Revisión administrativa
El administrador consume alertas y decide:

* descartar el evento
* reenviarlo a clasificación

Paso 7. Clasificación final
Los clasificadores toman eventos del buzón de clasificación y los redirigen según su tipo.

Paso 8. Procesamiento en servidores
Cada servidor consume eventos desde su buzón particular y simula procesamiento.

Paso 9. Terminación controlada

* El bróker envía fin al administrador.
* El administrador envía fines a clasificadores.
* El último clasificador envía fines a servidores.
* Main espera el cierre total con join().

7. **Sincronización entre objetos**

Uno de los requisitos más importantes del caso consiste en explicar cómo se sincronizan los objetos que interactúan entre sí.

7.1 Sensor → Buzón de Entrada:

Los sensores son productores concurrentes que comparten el mismo buzón.

Solución aplicada

* Método sincronizado de depósito.
* El bróker consume desde la cola.
* Al depositar se ejecuta notifyAll().

7.2 Broker – Buzón de Clasificación

Cuando el evento es normal, el bróker intenta depositarlo en un buzón limitado.
En el caso de que esté lleno:

* El bróker espera con wait().

En el caso de que haya espacio:

* Los clasificadores liberan capacidad y despiertan threads con notifyAll().

7.3 Broker – Buzón de Alertas
El bróker deposita alertas en una cola compartida que luego revisa el administrador.

7.4 Administrador – Buzón de Clasificación
El administrador también puede enviar eventos a clasificación. En la solución propuesta se usa espera semi-activa con Thread.yield() cuando el buzón está lleno.

7.5 Clasificador – Buzón de Clasificación
Varios clasificadores consumen desde una misma cola.
La operación de retiro sincronizado evita que dos clasificadores procesen el mismo evento.

7.6 Clasificador – Buzón de Consolidación
Cada clasificador produce eventos para el servidor correspondiente.

En el caso de que el buzón esté lleno:
* El clasificador espera con wait().

En el caso de que el servidor consuma:
* libera espacio y se despiertan productores.

7.7 Servidor – Buzón de Consolidación
Cada servidor consume solo de su propia cola.

En el caso de que la cola esté vacía:
* espera pasivamente con wait().

En el caso de que llegue un evento:
* el clasificador ejecuta notifyAll().

7.8 Clasificador – ControlFinClasificadores
Se usa un método sincronizado para descontar clasificadores activos y detectar cuál fue el último en terminar.

Esto evita condiciones de carrera.

7.9 Main – Todos los threads
La clase principal usa:

* CyclicBarrier para coordinar arranque.
* join() para esperar cierre total.

8. **Validación y pruebas realizadas**

Para verificar el correcto funcionamiento del sistema se utilizó el archivo de configuración:

```text
**ni=3**
**base=5**
**nc=5**
**ns=2**
**tam1=2**
**tam2=2**
```

Pruebas ejecutadas
Aclaración: Cuando se menciona **resultado**, significa respuesta que aparece en la terminal al ejecutar la prueba:

Prueba 1. Compilación
Se compiló el proyecto con:

```bash
javac -d out src\*.java
```
Resultado: compilación exitosa.

Prueba 2. Ejecución general
Se ejecutó con:

```bash
java -cp out Main
```
Resultado: el programa inició correctamente.

Prueba 3. Flujo de eventos
Se verificó visualmente que los eventos recorrieran todas las etapas:

* sensores
* bróker
* administrador
* clasificadores
* servidores

Prueba 4. Sin bloqueos

El sistema finalizó completamente sin deadlocks ni espera infinita.

Prueba 5. Terminación ordenada

Se confirmó que todos los threads terminaron correctamente después de recibir sus eventos de fin.

9. **Resultados obtenidos**

Durante las ejecuciones realizadas se observó:

* distribución correcta de eventos según tipo,
* procesamiento concurrente real,
* uso adecuado de buzones limitados,
* coordinación correcta entre threads,
* finalización ordenada de todo el sistema.

El comportamiento observado coincide con los objetivos planteados al inicio del proyecto tanto individualmente como grupalmente.

10. **Conclusiones**

Se implementó una arquitectura modular basada en múltiples hilos que cooperan entre sí mediante monitores y colas compartidas. Los mecanismos de sincronización utilizados garantizaron consistencia en los datos, control de acceso concurrente y coordinación adecuada entre procesos, previniendo así problemas relacionados con la concurrencia vistos en clase (desde condiciones de carrera hasta bloqueos en el sistema mientras se verifica un recorrido).

Adicionalmente, el desarrollo del caso permitió aplicar de manera práctica conceptos fundamentales de programación concurrente en Java, tales como la comunicación entre threads, el uso de recursos compartidos, la coordinación entre productores y consumidores y la finalización ordenada de procesos. La separación del sistema en módulos independientes facilitó tanto el trabajo colaborativo entre los integrantes como la integración progresiva de cada componente. De igual forma, las pruebas realizadas evidenciaron un comportamiento estable, una correcta distribución de eventos y una ejecución coherente de todas las etapas definidas en la solución.

En síntesis, el proyecto demuestra que una correcta división del problema en etapas concurrentes permite construir sistemas robustos y eficientes incluso en escenarios con múltiples productores y consumidores simultáneos.